INFORME DE LABORATORIO

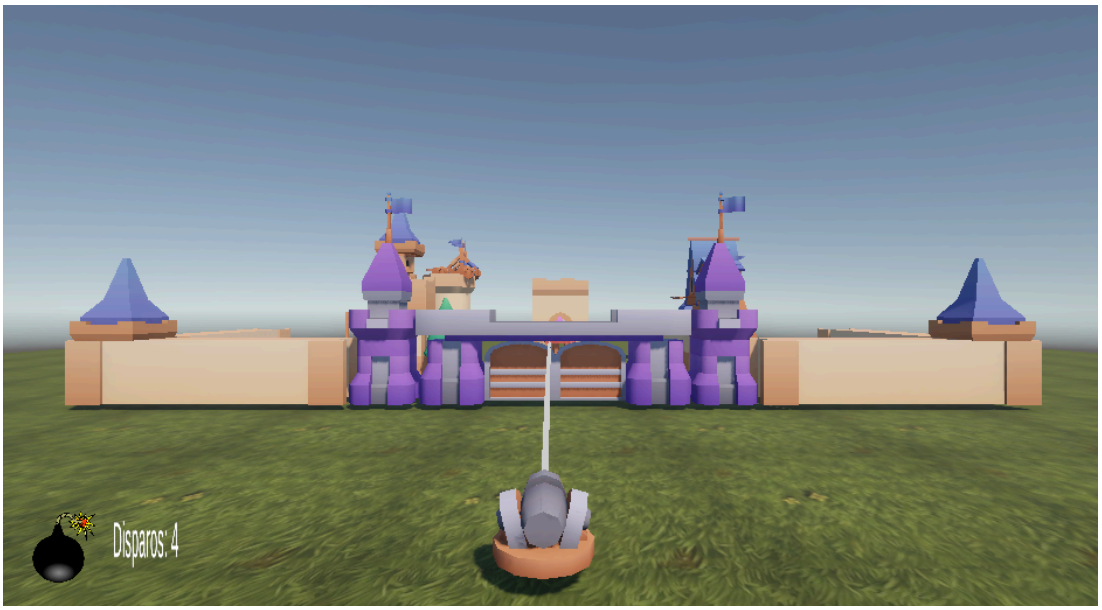
(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	*Proyectiles*				
NÚMERO DE PRÁCTICA:	*1*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2026-A*
FECHA DE PRESENTACIÓN	*03/05/20206*	**HORA DE PRESENTACION**	*23:59*		
INTEGRANTE (s): Chahua Charca, Luis Ayben				**NOTA:**	
DOCENTE(s): *Rene Alonso Nieto Valencia*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

1. ***Idea general del juego***
   El juego trata de un cañón que tiene que destruir la puerta o causar el máximo daño posible con las balas que tiene. el juego es simple, tienes una cantidad determinada de balas, si destruyes la puerta y las paredes alrededor de esta, ganarás, si te quedas sin balas perderas.



2. *Jugador y controles*

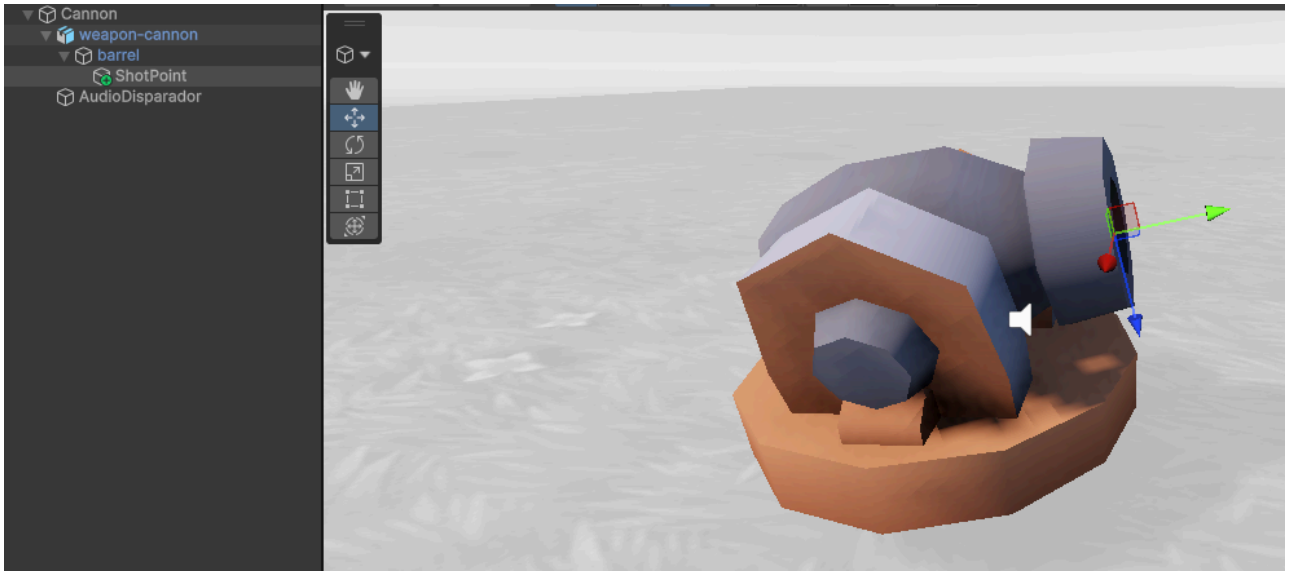
El jugador en un cañoncito que puede girar para apuntar su disparo, está controlado por el script CannonController, que le permite apuntar, contar la cantidad de disparos que le quedan, afectar el UI del juego en base a sus proyectiles y guarda los valores de puntaje también.
para los controles, se separa el cañón de su barril, cuando gira verticalmente gira solo el barril, y cuando lo hace horizontalmente gira todo el cañón completo.

```C/C++
public class CannonController : MonoBehaviour
{
    // VARIABLES AQUÍ (dentro de la clase)
    public Transform baseCannon;
    public Transform barrel;

    public float rotationSpeed = 1500f;

    public float BlastPower = 5;

    public GameObject Cannonball;
    public Transform ShotPoint;

    private float verticalAngle = 0f;
    public float minAngle = 0;
    public float maxAngle = 40;

    private float horizontalAngle = 0f;
    public float minHorizontal = -60f;
    public float maxHorizontal = 60f;

    [Header("Audio Settings")]
    [SerializeField] public AudioSource disparador;
    public AudioClip disparo;

    [Header("Disparos Disponibles")]
    [SerializeField] public int ProyectilesDisponibles = 4;



    //  Declarar correctamente el evento como miembro de la clase
    public System.Action onSinDisparos;

    void Update()
    {
        Rotar();
        if (Input.GetKeyDown(KeyCode.Space))
        {
            if (ProyectilesDisponibles > 0)
            {
                StartCoroutine(Disparar());
```

```
            }
        }
    }

    private IEnumerator Disparar()
    {
        disparador.PlayOneShot(disparo);

        yield return new WaitForSeconds(0.3f);

        GameObject ball = Instantiate(Cannonball, ShotPoint.position,
ShotPoint.rotation);
        Rigidbody rb = ball.GetComponent<Rigidbody>();
        rb.linearVelocity = ShotPoint.up * BlastPower;

        ProyectilesDisponibles--;

        if (ProyectilesDisponibles <= 0)
        {
            onSinDisparos?.Invoke();
        }
    }


    private void Rotar()
    {
        //  ROTACIÓN HORIZONTAL (base del cañón)
        if (Input.GetKey(KeyCode.A))
        {
            horizontalAngle -= rotationSpeed * Time.deltaTime;
        }

        if (Input.GetKey(KeyCode.D))
        {
            horizontalAngle += rotationSpeed * Time.deltaTime;
        }

        // Limitar ángulo horizontal
        horizontalAngle = Mathf.Clamp(horizontalAngle, minHorizontal, maxHorizontal);

        // Aplicar rotación absoluta
        baseCannon.localRotation = Quaternion.Euler(0, horizontalAngle, 0);

        //  ROTACIÓN VERTICAL (tubo del cañón)
        if (Input.GetKey(KeyCode.W))
        {
            verticalAngle -= rotationSpeed * Time.deltaTime;
        }

        if (Input.GetKey(KeyCode.S))
```

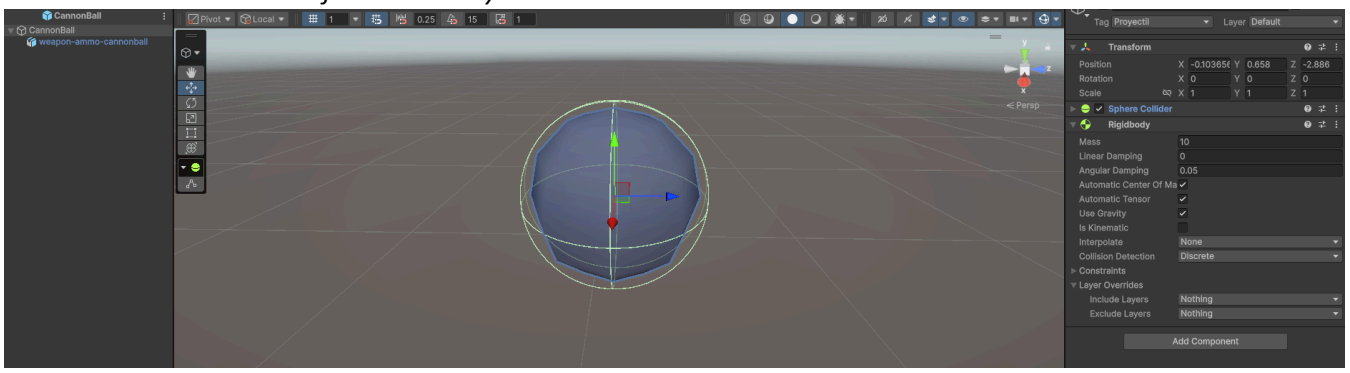
```
        {
            verticalAngle += rotationSpeed * Time.deltaTime;
        }

        // Limitar ángulo
        verticalAngle = Mathf.Clamp(verticalAngle, minAngle, maxAngle);

        // Aplicar rotación al tubo
        barrel.localRotation = Quaternion.Euler(verticalAngle, 0, 0);
    }

}
```

El cañón está compuesto por el asset y un shot point que controlará la dirección en la que se instancian las balas de cañón y comienzan su trayectoria tras el disparo.



3. ***Proyectil***

   El proyectil es simplemente una esfera con textura que se instala para que pueda ser expulsada, tiene un rigidbody y un collider para trabajar en este laboratorio, ya que son útiles para detectar colisiones e interactuar con las físicas en unity.



4. ***Trayectoria***

En el juego existe una linea blanca que marca la trayectoria que el objeto proyectil va a seguir una vez disparado, esto se controla con el Script "Draw Projection"

C/C++

```
using System.Collections.Generic;
using UnityEngine;

public class Draw : MonoBehaviour
{
    CannonController cannonController;
    LineRenderer lineRenderer;

    public int numPoints = 50;

    public float timeBetweenPoints = 0.02f;

    public LayerMask CollidableLayers;

    // Start is called once before the first execution of Update after the MonoBehaviour
is created
    void Start()
    {
        cannonController = GetComponent<CannonController>();
        lineRenderer = GetComponent<LineRenderer>();
    }

    // Update is called once per frame
    void Update()
    {
        lineRenderer.positionCount = numPoints;

        List<Vector3> points = new List<Vector3>();

        Vector3 startingPosition = cannonController.ShotPoint.position;
        Vector3 startingVelocity = cannonController.ShotPoint.up *
cannonController.BlastPower;

        Vector3 previousPoint = startingPosition;

        for (int i = 0; i < numPoints; i++)
        {
            float t = i * timeBetweenPoints;

            Vector3 newPoint = startingPosition
                             + startingVelocity * t
                             + 0.5f * Physics.gravity * t * t;

            // DETECCIÓN DE COLISIÓN REAL
```

```
            if (Physics.Raycast(previousPoint, newPoint - previousPoint, out RaycastHit
hit,
                                (newPoint - previousPoint).magnitude, CollidableLayers))
            {
                points.Add(hit.point); // punto exacto de impacto
                break;
            }

            points.Add(newPoint);
            previousPoint = newPoint;
        }

        lineRenderer.positionCount = points.Count;
        lineRenderer.SetPositions(points.ToArray());
    }
}
```

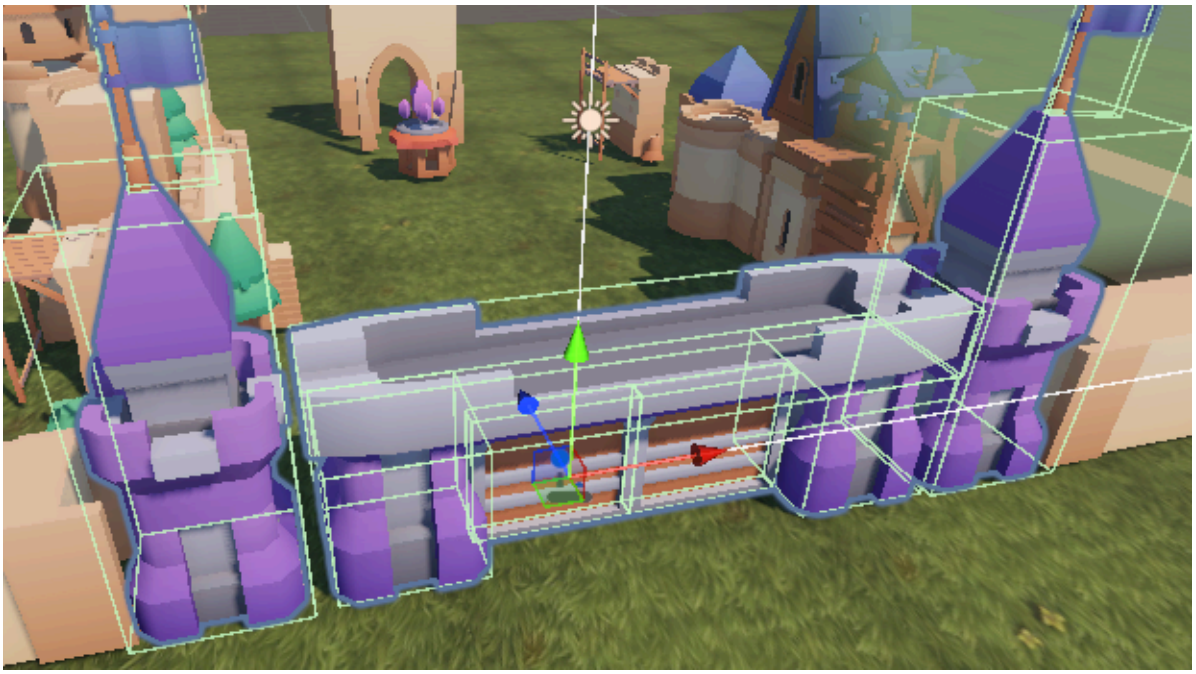
Que recoge las referencias del cañón y su shotpoint, para mediante eso y un componente Line Renderer, calcular con la fórmula de movimiento compuesto en los ejes x, y
para 50 puntos y marcas de tiempo calculadas cada 0.02 segundos.

$$x = x_0 + v_x \cdot t$$

$$y = y_0 + v_{0y} \cdot t + \frac{1}{2} \cdot a_y \cdot t^2$$

5. Detección de colisiones

Para el caso de las colisiones, estas se controlan en un script asignado a cada objeto destructible del mapa



Cada elemento cuenta con el suyo propio, que avisa al gameManager que han sido destruidos (Impactados y movidos de sus lugares) para otorgar puntaje o victoria dado sea el caso de la colisión.

6. ***Victoria, y puntaje***

Se controla mediante el Game Controller, que permite gestionar la victoria, derrota y todo de la UI, además es el que recibe los cambios en los ítems destructibles, para así dar la victoria o la derrota.

```
C/C++
sing UnityEngine;
using TMPro;

public class GameManager : MonoBehaviour
{
    public static GameManager instance;
    [Header("Triggers")]
    [SerializeField] CannonController cannonController;

    [Header("UI Juego")]
    [SerializeField] GameObject canvasJuego; // HUD normal

    [Header("UI Resultado")] // fondo (imagen o panel)
    [SerializeField] Canvas textoGanaste;
    [SerializeField] Canvas textoPerdiste;

    private int secundariosDerribados = 0;
    [SerializeField] int necesariosParaGanar = 5;
    private bool juegoTerminado = false;
```

```
[Header("Puntaje")]
private int puntaje = 0;
[SerializeField] TextMeshProUGUI textoPuntaje;
[SerializeField] int puntosSecundario = 100;
[SerializeField] int puntosPrincipal = 200;
[SerializeField] int puntosPorDisparoRestante = 200;

void Awake()
{
    instance = this;
}

void Start()
{
    // Estado inicial
    textoPuntaje.gameObject.SetActive(true);
    textoGanaste.gameObject.SetActive(false);
    textoPerdiste.gameObject.SetActive(false);
    if (cannonController != null)
    {
        cannonController.onSinDisparos += GameOver;
    }
}

public void GameOver()
{

    canvasJuego.SetActive(false);

    textoPerdiste.gameObject.SetActive(true);
    textoGanaste.gameObject.SetActive(false);

    Time.timeScale = 0f;
}

public void Win()
{

    canvasJuego.SetActive(false);

    int disparosRestantes = cannonController.ProyectilesDisponibles;
    puntaje += disparosRestantes * puntosPorDisparoRestante;

    textoGanaste.gameObject.SetActive(true);
    textoPerdiste.gameObject.SetActive(false);

    Time.timeScale = 0f;
}

public void RegistrarSecundario()
```

```
    {
        if (juegoTerminado) return;
        secundariosDerribados++;
        if (secundariosDerribados >= necesariosParaGanar)
        {
            juegoTerminado = true;
            Win();
        }
    }

    public void SumarSecundario()
    {
        puntaje += puntosSecundario;
        RegistrarSecundario();
        ActualizarPuntaje();
    }

    public void SumarPrincipal()
    {
        puntaje += puntosPrincipal;
        ActualizarPuntaje();
        Win();
    }

    public void ActualizarPuntaje()
    {
        textoPuntaje.text = "Puntos: " + puntaje;
        Debug.Log("Puntaje actualizado: " + puntaje);
    }
}
```

II. SOLUCIÓN DEL CUESTIONARIO

III. CONCLUSIONES

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA